

VR Cryptic Hunt

Project Team

Khuzaima Sajjad	19I-2201
Umer Minhas	20I-2331
Bilal Farooqi	20I-1788

Session 2021-2025

Supervised by

Ms. Rabail Zahid

Co-Supervised by

Mr. Saad Salman



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1 : Environment and asset creation	2
1.3.2	Module 2 : Assets and environment integration	3
1.3.3	Module 3 : Sound and audio design	3
1.3.4	Module 4 : UI and settings	3
1.3.5	Module 5 : Treasure interaction and puzzle	3
1.3.6	Module 6 : Scaling of difficulty levels	4
1.3.7	Module 7 : Integration of VR	4
1.3.8	Module 8 : Testing	4
1.3.9	Module 9 : Optimization and deployment	4
1.4	User Classes and Characteristics	4
2	Project Requirements	7
2.1	Use-case	7
2.1.1	Highlevel usecase	8
2.1.1.1	Use case 1: Explore the island	8
2.1.1.2	Use case 2: Find treasure chest	8
2.1.1.3	Use case 3: Solve puzzle to unlock chest	8
2.1.1.4	Use case 4: Gather survival items	8
2.1.1.5	Use case 5: Climb Mountain	8
2.1.1.6	Use case 6: Interact with environment	8
2.1.1.7	Use case 7: Avoid hazards	9
2.1.1.8	Use case 8: Access Map or Inventory	9
2.1.1.9	Use case 9: Rest or Set Up Camp	9
2.1.1.10	Use case 10: Complete the scavenger hunt	9
2.1.2	Expanded Usecase	9
2.1.2.1	ID: UC1	9
2.1.2.2	ID: UC2	10
2.1.2.3	ID: UC3	11

2.1.2.4	ID: UC4	11
2.1.2.5	ID: UC5	12
2.1.2.6	ID: UC6	12
2.1.2.7	ID: UC7	13
2.1.2.8	ID: UC8	14
2.1.2.9	ID: UC9	14
2.1.2.10	ID: UC10	15
2.2	Functional Requirements	16
2.2.1	Module 1: Environment and asset creation	16
2.2.2	Module 2: Assets and environment integration	16
2.2.3	Module 3: Sound and audio design	16
2.2.4	Module 4: UI and settings	16
2.2.5	Module 5: Treasure interaction and puzzle	17
2.2.6	Module 6: Scaling of difficulty levels	17
2.2.7	Module 7: Integration of VR	17
2.2.8	Module 8: Testing	17
2.2.9	Module 9: Optimization and deployment	18
2.3	Non-Functional Requirements	18
2.3.1	Reliability	18
2.3.2	Usability	18
2.3.3	Performance	18
2.3.4	Security	19
2.4	Domain Model	19
3	System Overview	23
3.1	Architectural Design	23
3.2	Design Models	23
3.2.1	Activity Diagram	25
3.2.2	Class Diagram	26
3.2.3	Class-level Sequence Diagram	26
3.2.3.1	Sequence diagram 1	26
3.2.3.2	Sequence diagram 2	27
3.2.3.3	Sequence diagram 3	27
3.2.3.4	Sequence diagram 4	28
3.2.3.5	Sequence diagram 5	29
3.2.3.6	Sequence diagram 6	30
3.2.3.7	Sequence diagram 7	31
3.2.3.8	Sequence diagram 8	32
3.2.3.9	Sequence diagram 9	33
3.3	Data Design	33

3.3.1	Major Data Entities	34
3.3.1.1	Player	34
3.3.1.2	Item	34
3.3.1.3	Puzzle	34
3.3.1.4	TreasureChest	34
3.3.1.5	Hazard	34
3.3.2	Data Storage and Processing	35
3.3.2.1	Database Management System	35
3.3.3	Data Organization	35

List of Figures

List of Tables

Chapter 1

Introduction

The project is a virtual reality treasure hunt game where players solve puzzles to uncover encrypted valuables while exploring a 3D world. It's intended to provide an immersive and interesting experience. The game, which was created with Unity, mixes the mental challenge of solving puzzles with the thrill of treasure hunting. To access the goodies inside, players must travel the virtual environment, find secret chests, and work simple puzzles. The goal of the game is to give players of all skill levels—from casual gamers to VR enthusiasts—an enjoyable, approachable, and participatory virtual reality experience. Focusing on user-friendly controls, adaptable difficulty settings, and an open-world setting, the project also demonstrates inventive VR technology utilization while guaranteeing usability and scalability for future improvements. This project offers useful insights on the creation of VR applications and interactive user experiences in addition to being an entertaining game.

1.1 Problem Statement

Modern technology may offer an fascinating experience and instructional value that traditional scavenger hunt games cannot match. Furthermore, they frequently fall short in encouraging users to think critically and solve problems. A more demanding and dynamic kind of entertainment that not only keeps players interested but also improves their cognitive abilities is required in the digital age. By creating a Virtual Reality scavenger hunt game where players explore an intricately detailed virtual world, solve cryptographic riddles, and decipher hidden prizes, our project seeks to close this gap. The game will provide a distinctive fusion of learning and fun by fusing the excitement of exploration with the instructional advantages of cryptography.

1.2 Scope

The goal of this project is to create a puzzle-solving and exploration-based virtual reality game that will captivate players in a 3D world while they search for hidden treasure chests that are encrypted. To unlock the contents of a chest, the player must solve a straightforward puzzle, which can be found inside the chest. The primary features of the VR environment include movement, interactions with objects, and an easy-to-use user interface. Additionally, the project will have varying degrees of difficulty, enabling a steady rise in the intricacy of the riddles and the locations of the hidden jewels. This scalability makes sure that players with different skill levels may still enjoy the game and challenges those who are more experienced. The game's immersive quality will also be improved by visual and aural enhancements, with special focus paid to performance optimisation for seamless VR operation. The project's limitations include creating a wellorganised gaming environment, putting in place fundamental VR controls, and making sure that riches are encrypted. Advanced multiplayer capabilities and extremely intricate encryption schemes, however, are outside the purview of this document. The project's main goal will be to produce a strong, captivating single-player game with the potential for expansion in the future. As development progresses, the game's scalable design will enable future updates, new stages, and increasingly challenging riddles. The end product will be a well-executed, functional VR game that showcases both technological expertise and imaginative design.

1.3 Modules

The virtual reality scavenger hunt game's functionality and structure are specified by a number of modules that make up the proposed project. Every module will concentrate on particular game aspects, guaranteeing fluid game play and an enjoyable user experience. The modules are organised based on the functionalities of the system, which includes puzzle-solving, treasure decryption, and VR environment user interaction.

1.3.1 Module 1 : Environment and asset creation

In this module, the 3D setting where the scavenger hunt will take place is designed and developed. Creating the world's inhabitants, such as buildings, trees, chests, and other interacting items, is another aspect of it.

1. Create a dynamic environments.
2. Produce assets such as food, animals and character

1.3.2 Module 2 : Assets and environment integration

The environments and assets from the previous module are combined into a unified universe in this module. This guarantees seamless communication between the various items and their surroundings.

1. Include 3D elements in the area that has been designed.
2. Verify that the items interact, scale, and align correctly.

1.3.3 Module 3 : Sound and audio design

The incorporation of sound into the game is managed by this module. It emphasises enhancing immersion through the use of ambient audio, sound effects for interactions, and background music.

1. Create required sounds if necessary
2. Include ambient noises and background music.
3. Add sound effects to objects that interact with them.

1.3.4 Module 4 : UI and settings

In order to make the game's menus and settings easy for players to traverse, this module focuses on designing an intuitive user interface. It also entails configuring the settings for things like sound and difficulty levels.

1. Make menus for the game, settings, and tutorials.
2. Offer options to change the game's difficulty, visual quality, and volume.
3. Verify that the menus and navigation options are working correctly

1.3.5 Module 5 : Treasure interaction and puzzle

Treasure interactions are introduced in this module, where players find hidden chests and have to solve puzzles to open them. These are the tools required to reach the next destination.

1. Create treasure boxes with required tools hidden in it.
2. Encrypt the boxes.
3. Create easy-to-solve riddles to access the tools

1.3.6 Module 6 : Scaling of difficulty levels

As the player advances in this module, the puzzles and treasure locations get more difficult. It guarantees that even at higher levels, the game will be difficult and captivating.

1. As the player advances, the puzzle difficulty will gradually get harder.
2. Change the treasures' locations and encryption to increase the difficulty of finding them.
3. Create a plane and a boat as the final mission for the player to get off the island.

1.3.7 Module 7 : Integration of VR

In order to enable players to engage with the game in a completely immersive manner, this module focuses on integrating the game with virtual reality.

1. Include VR controllers and object interactions in games.
2. Adjust the game to run well in the virtual reality setting.

1.3.8 Module 8 : Testing

This module contains the game's final testing to find and remove bugs.

1. Conduct thorough testing of all VR interactions and game mechanics.
2. Remove any bugs found to make the game bug free, scale, and align correctly.

1.3.9 Module 9 : Optimization and deployment

This module deals with the optimization of the game which will guarantee seamless game play and top performance across several platforms and it also deals with the deployment phase.

1. Optimize the game to make it compatible and playable across all platforms.
2. Deploy the game.

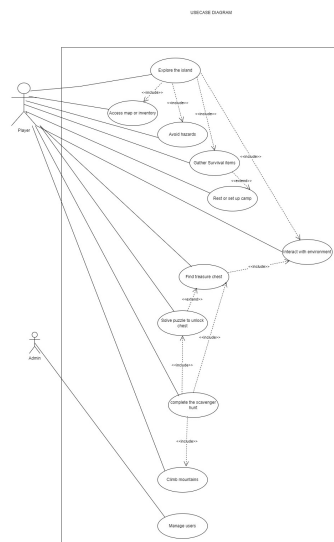
1.4 User Classes and Characteristics

User class	Description
Player	The main user interacting with the virtual environment is the player. To advance, they must investigate the island, work out puzzles, and gather things. To overcome obstacles and find wealth, the player must make use of a variety of interactive tools and movement controls. Their differing degrees of gaming experience have an impact on their capacity to solve riddles and advance the game.
Game Developer	The scavenger hunt game must be created, maintained, and updated by the game developer. They oversee the game's mechanics, fix bugs, and guarantee an engaging gaming experience. They are responsible for developing environments, coming up with puzzles, and testing the game to make sure everything works properly. They need to be technically proficient in both game design and VR creation.
Administrator	The administrator oversees system updates and keeps an eye on player data. They maintain server-side features like player progress saving and database management, making ensuring the game is safe and functional. Along with handling user administration, administrators also look after player account integrity and offer technical help.
Tester	The tester interacts with the game during its development and testing phases to guarantee functioning and usability. To uncover flaws and make suggestions for improvements, they carry out a variety of test cases, such as locating treasure chests, navigating the terrain, and solving riddles. Their opinions are very important to improve the user interface and game mechanics.

Chapter 2

Project Requirements

2.1 Use-case



usecase diagram

2.1.1 Highlevel usecase

2.1.1.1 Use case 1: Explore the island

Actor: Player Goal: Explore the dynamic environment. Description: The player explores the island's beach, mountains, and forest in quest of interactive features and hidden treasures.

2.1.1.2 Use case 2: Find treasure chest

Actor: Player Goal: Locate a hidden treasure chest. Description: When the player gets to a certain location, they find a treasure chest that they can open to reveal useful things.

2.1.1.3 Use case 3: Solve puzzle to unlock chest

Actor: Player Goal: Solve a puzzle to open the treasure chest. Description: The player has to figure out a basic problem in order to try to open a treasure chest. When the problem is successfully solved, the chest's contents will become visible.

2.1.1.4 Use case 4: Gather survival items

Actor: Player Goal: Collect food and survival tools. Description: Food and tools that are essential for survival are gathered by the player as they explore the island and are kept in their inventory for later use.

2.1.1.5 Use case 5: Climb Mountain

Actor: Player Goal: Navigate mountain terrain to reach the beach. Description: To reach various places of the island, including those that are concealed or otherwise inaccessible, the player must ascend difficult mountain slopes.

2.1.1.6 Use case 6: Interact with environment

Actor: Player Goal: Interact with objects in the environment. Description: Through interaction with different things (trees, rocks, etc.), the player can uncover items, hidden passageways, or clues for the scavenger hunt.

2.1.1.7 Use case 7: Avoid hazards

Actor: Player Goal: Avoid natural hazards in the environment. Description: In order to avoid getting hurt or having their quest delayed, the player must safely maneuver around deadly hazards like falling rocks and high cliffs.

2.1.1.8 Use case 8: Access Map or Inventory

Actor: Player Goal: View the island map or check inventory. Description: The player can handle the tools and items they have obtained by opening their inventory or by viewing a virtual map that shows them where they are on the island.

2.1.1.9 Use case 9: Rest or Set Up Camp

Actor: Player Goal: Regain health and energy by resting. Description: After locating a safe area, the player sets up a makeshift camp or shelter using the rest feature, which restores their health and gives them more energy for exploration.

2.1.1.10 Use case 10: Complete the scavenger hunt

Actor: Player Goal: Find and unlock all treasure chests to complete the game. Description: The player finishes the treasure hunt and advances to the game's ultimate objective after finding and opening every treasure chest on the island.

2.1.2 Expanded Usecase**2.1.2.1 ID: UC1**

Name: Explore the Island

Author: Khuzaima Sajjad

Description: The player explores the island environment to locate hidden treasures and interactive items to survive.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player has started the game and is in the island environment.

Main Flow:

1. The player's entry into the VR environment initiates the use case.

2. The island is displayed by the system with different regions (beach, jungle, mountains).
3. The player uses movement controls to investigate the surroundings by navigating the environment.
4. The player explores locations that might contain hidden treasures and engages with surrounding items.
5. The system responds to the player's interactions by pointing out hidden routes or providing hints.

Post Conditions: The player successfully explores the environment, uncovering new areas or clues.

Alternative Flows: If the player encounters hazards or blocked paths, they must find alternate routes to continue exploration.

2.1.2.2 ID: UC2

Name: Find Treasure Chest

Author: Khuzaima Sajjad

Description: The player locates a treasure chest hidden in the environment.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is actively exploring the island environment.

Main Flow:

1. The use case begins when the player finds a treasure chest in the surroundings, either partially or completely visible.
2. The chest is emphasized by the system as an interacting item.
3. The player moves closer to the chest and engages with it.
4. The player is prompted by the system to use a riddle or clue to unlock the chest.

Post Conditions: The player successfully finds the treasure chest and can proceed to the unlocking phase.

Alternative Flows: If the player overlooks the chest or fails to interact, they may need to revisit the area to attempt interaction again.

2.1.2.3 ID: UC3

Name: Solve Puzzle to Unlock Chest

Author: Khuzaima Sajjad

Description: The player solves a simple puzzle to unlock the treasure chest.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player has found the chest and is attempting to open it.

Main Flow:

1. When the player tries to open the treasure chest, that is when the use case begins.
2. The player is given an obstacle by the system to complete, such as matching patterns or deciphering a riddle.
3. To complete the problem, the player must interact with its elements.
4. The system will unlock the chest and reveal the riches within if the puzzle is correctly answered.

Post Conditions: The player successfully opens the chest and retrieves its contents.

Alternative Flows: If the player fails the puzzle, they may retry or search for additional clues.

2.1.2.4 ID: UC4

Name: Gather Survival Items

Author: Khuzaima Sajjad

Description: The player collects survival tools and food to aid in their journey.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is exploring the environment and finds survival items.

Main Flow:

1. When the player finds a survival item (food, tools, etc.), the use case begins.
2. The item is marked as an interactive object by the system.

3. To obtain the object, the player must interact with it.
4. The item is added to the player's inventory by the system.

Post Conditions: The survival item is successfully stored in the player's inventory.

Alternative Flows: If the player does not pick up the item, it remains in the environment for future collection.

2.1.2.5 ID: UC5

Name: Climb Mountain

Author: Khuzaima Sajjad

Description: The player navigates the mountain terrain to access new areas.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is at the base of the mountain and ready to climb.

Main Flow:

1. When the player arrives at the mountain's base, the use case begins.
2. The player must navigate a path or climb a challenge presented by the system.
3. To climb the mountain, the player engages with the surroundings.
4. The player can explore new regions of the island after reaching the summit.

Post Conditions: The player successfully climbs the mountain and unlocks new exploration opportunities.

Alternative Flows: If the player fails to navigate the mountain, they may need to find alternate routes or retry the climb.

2.1.2.6 ID: UC6

Name: Interact with Environment

Author: Umer Minhas

Description: The player interacts with objects in the environment to uncover clues or hidden items.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is in the environment and finds an interactive object.

Main Flow:

1. When the player comes across an interactive object (such as a tree, rock, etc.), the use case begins.
2. The object for interaction is highlighted by the system.
3. The object can expose hidden things or hints when the player interacts with it.
4. The system gives you feedback by opening up more locations or supplying more hints.

Post Conditions: The player interacts with the environment, gaining additional insights or resources.

Alternative Flows: If the player misses interactive objects, they may have to retrace their steps to find them.

2.1.2.7 ID: UC7

Name: Avoid Hazards

Author: Bilal Farooqi

Description: The player encounters and avoids natural hazards in the environment.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is exploring areas with potential hazards.

Main Flow:

1. The use case begins when the player steps into a hazardous environment (such as a high slope or falling pebbles).
2. The system indicates when dangers are there and offers ways to get away.
3. The player moves through the dangers without getting hurt.
4. The player is permitted to proceed after the system recognizes their effective avoidance.

Post Conditions: The player avoids the hazard and continues exploring.

Alternative Flows: If the player fails to avoid the hazard, they may take damage or be forced to restart the area.

2.1.2.8 ID: UC8

Name: Access Map or Inventory

Author: Bilal Farooqi

Description: The player checks their inventory or map to plan their next move.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player is in-game and needs to check inventory or location.

Main Flow:

1. When the player opens the inventory screen or map, the use case begins.
2. A map of the player's location or inventory with the objects they have collected is displayed by the system.
3. The player looks up their location or what's in their inventory.
4. When a player leaves, the system closes the inventory or map.

Post Conditions: The player gains valuable information about their location or available tools. Alternative Flows: If the player does not access the map or inventory, they may rely on exploration without guidance.

2.1.2.9 ID: UC9

Name: Rest or Set Up Camp

Author: Umer Minhas

Description: The player rests to regain energy and health by setting up a camp.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player finds a safe location in the environment.

Main Flow:

1. When the player arrives at a secure area, the use case begins.
2. The opportunity to recuperate by resting is provided by the system.
3. The player erects a makeshift shelter or camp.
4. The player recovers energy from their rest and carries on with their quest.

Post Conditions: The player's health and energy are replenished.

Alternative Flows: If the player does not rest, they may struggle to continue their journey with low health.

2.1.2.10 ID: UC10

Name: Complete the Scavenger Hunt

Author: Umer Minhas

Description: The player completes the scavenger hunt by finding and unlocking all treasure chests.

Primary Actors: Player

Secondary Actors: None

Preconditions: The player has found and unlocked most or all treasure chests.

Main Flow:

1. When the player locates the last treasure chest, the use case begins.
2. The system offers a last challenge or puzzle and verifies the location of the chest.
3. After solving the last problem, the player opens the chest.
4. The player wins the game when the algorithm declares the scavenger hunt to be finished.

Post Conditions: The player successfully completes the scavenger hunt and is rewarded.

Alternative Flows: If the player fails to complete all chests, they may replay the game or continue exploring.

2.2 Functional Requirements

2.2.1 Module 1: Environment and asset creation

1. The system shall create different 3D environments on the fly using preset parameters for the scavenger quest.
2. The system shall generate and store necessary resources like food, animals, and characters to animate the surroundings.
3. To improve the immersive experience, the system shall make sure that all created assets have the right size, texture, and realism.

2.2.2 Module 2: Assets and environment integration

1. To ensure smooth rendering of all assets in the world, the system shall include the produced 3D elements into the game environment.
2. The system shall confirm that the environment's objects interact correctly with one another, with accurate scaling and alignment.
3. The system shall make sure that when a player interacts with trees, buildings, and chests, the environment will react accordingly.

2.2.3 Module 3: Sound and audio design

1. To improve player immersion, the system shall permit the installation of background music, sound effects, and ambient noises.
2. The system shall have sound effects when interacting with particular items, such as animals, treasure chests, or puzzle-solving moments.
3. The system shall provide dynamic volume adjustment in response to certain interactions or objects' proximity.

2.2.4 Module 4: UI and settings

1. The system shall have a menu system for interacting with tutorials, settings, and gaming choices.

2. The system shall let players choose the game's volume, visual quality, and difficulty settings.
3. The system shall make sure that the UI components work properly and are responsive on different VR platforms.

2.2.5 Module 5: Treasure interaction and puzzle

1. The mechanism shall produce treasure chests filled with the tools needed to advance the game.
2. Treasure boxes shall be encrypted by the system, and players shall need to solve puzzles to access them.
3. As the player advances, the algorithm shall provide solvable riddles or puzzles that get more difficult.

2.2.6 Module 6: Scaling of difficulty levels

1. As the player progresses through the game, the system shall dynamically modify the difficulty of the riddles and treasure placements.
2. To generate increasingly difficult tasks at higher levels, the system shall alter treasure locations and encryption techniques.
3. In order to finish the game, the system shall introduce final mission tasks, such as building a boat or airplane.

2.2.7 Module 7: Integration of VR

1. 1: The system shall be completely integrated with virtual reality platforms so that users can use VR controllers to interact with the surroundings.
2. The technology shall guarantee that there is no discernible lag or performance problems while the game is played in a virtual reality setting.

2.2.8 Module 8: Testing

1. The entire system, including VR interfaces, game mechanics, and user experiences, shall be thoroughly tested.

2. There should be no bugs in the system, and any that are found should be addressed before it is released.

2.2.9 Module 9: Optimization and deployment

1. To ensure seamless operation on a range of VR gear, the system shall be optimized for numerous platforms.
2. Following optimization, the system shall launch the game on the designated platforms and make it playable.

2.3 Non-Functional Requirements

2.3.1 Reliability

The system is built to be dependable, with few disruptions to its continuous operation. To be more precise, the game must have a mean time between failures (MTBF) of at least 100 hours of gaming in order to allow people to play for longer stretches of time without experiencing serious bugs. An automated save feature will be incorporated to safeguard the player's progress and avert any data loss in the case of a system crash, ensuring a seamless playing experience.

2.3.2 Usability

USE-1: The system shall have an intuitive menu for users to access the settings, inventory, and map.

USE-2: To assist novice players in understanding the controls and goals of the game, the system shall provide tutorial prompts at the beginning of the game.

USE-3: Players shall be able to pause and continue the VR scavenger hunt without losing their progress.

2.3.3 Performance

PER-1: When switching between sections, the game shall load each scene in five seconds or less.

PER-2: To guarantee fluid gameplay, the game shall keep a minimum frame rate of 60 frames per second (FPS).

PER-3: Without sacrificing performance, the system shall be able to handle at least ten interacting objects in the player's field of vision at once.

2.3.4 Security

For the system to safeguard player data and game integrity, security is essential. To avoid unwanted access, all player data—including progress and inventory—must be encrypted and kept in a safe location. Furthermore, stringent security mechanisms will be implemented by the system to safeguard private in-game information, including secret goods or puzzle answers, so that outside manipulation cannot uncover these components. In order to enhance system security, the game will only be accessible to registered players, and all player data will be safely stored in a secured database.

2.4 Domain Model

Environment

- **Attributes:**
 - environmentID: Unique id for the environment.
 - description: Description of the environment.
 - assets: List of items in the environment.
 - hazards: List of hazards in the environment.
- **Relationships:**
 - Contains TreasureChest
 - Contains Item
 - Contains Hazard
 - Interacts with Player

Player

- **Attributes:**

- `playerID`: Unique id for the player.
- `name`: character's name.
- `currentLocation`: The current location of the player in the environment.
- `inventory`: List of items held by the player.
- `score`: Points earned by the player through achievements.
- `level`: Current level in the game.

- **Relationships:**

- Interacts with `TreasureChest`
- Interacts with `Puzzle`
- Uses `Item`
- Navigates within `Environment`
- Avoids `Hazard`

Item

- **Attributes:**

- `itemID`: Unique id for the item.
- `name`: Name of the item.
- `type`: Type of the item (e.g., consumable, collectible).
- `isUsable`: Bool indicating if the user can use the item.

- **Relationships:**

- Belongs to `Inventory`
- Used by `Player`
- Interacts with `Environment`

Puzzle

- **Attributes:**

- puzzleID: Unique id for the puzzle.
- description: Description of the puzzle.
- difficulty: Difficulty level of the puzzle.
- solution: The solution to solve the puzzle.

- **Relationships:**

- Associated with TreasureChest
- Interacted with by Player

TreasureChest

- **Attributes:**

- chestID: Unique id for treasure chest.
- location: The location of the chest.
- isLocked: Boolean indicating if the chest is locked.
- contents: List of items in the chest.

- **Relationships:**

- Contains Puzzle
- Interacts with Player
- Located in Environment

Hazard

- **Attributes:**

- hazardID: Unique id for the hazard.
- type: Type of hazard.
- damage: Amount of damage inflicted by the hazard.

- **Relationships:**

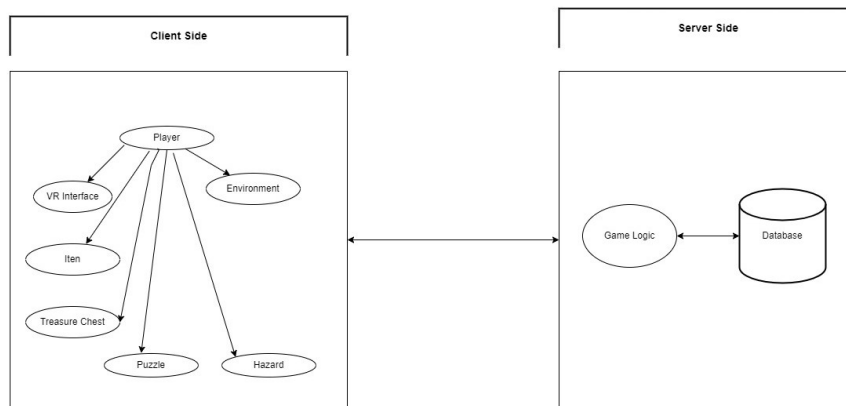
- Present in Environment
- Avoided by Player

Chapter 3

System Overview

3.1 Architectural Design

This game's architecture uses an object-oriented, modular design that is client-server in nature. By assigning a defined task to each component (or class), scalability, maintainability, and debugging simplicity of use are encouraged.



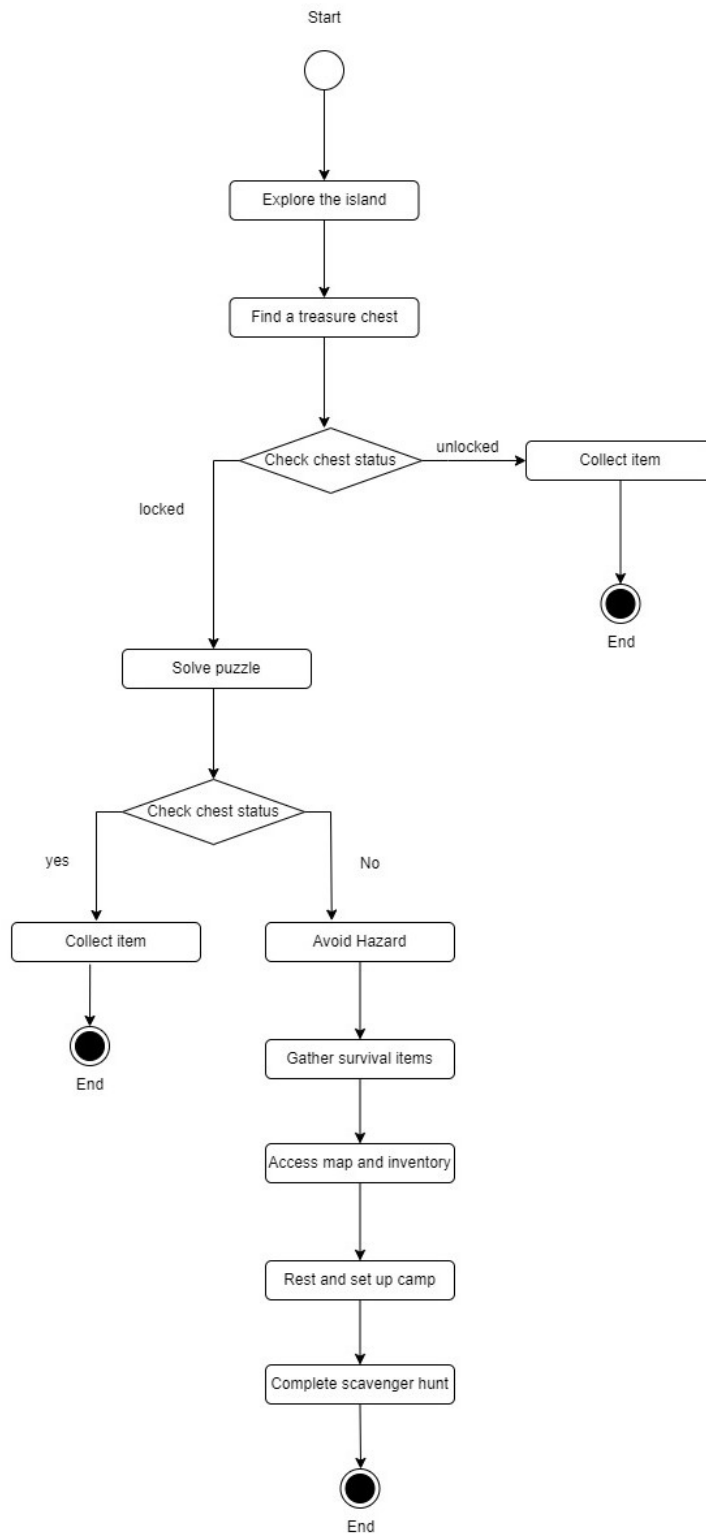
Architecture diagram

3.2 Design Models

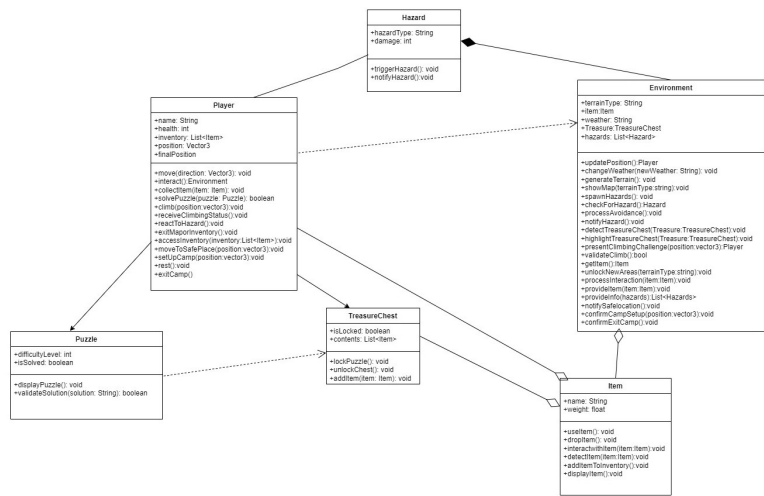
For this project, We have opted for an object-oriented paradigm to improve component re-usability and enable modular design. This strategy facilitates more lucid linkages be-

tween the many components of the system and enables better control of intricate interactions. The applicable models for the project using object oriented development approach include:

3.2.1 Activity Diagram



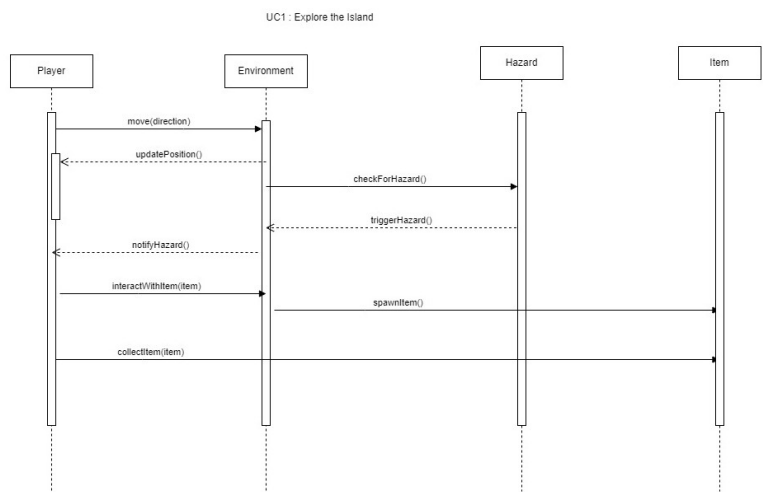
3.2.2 Class Diagram



Class diagram

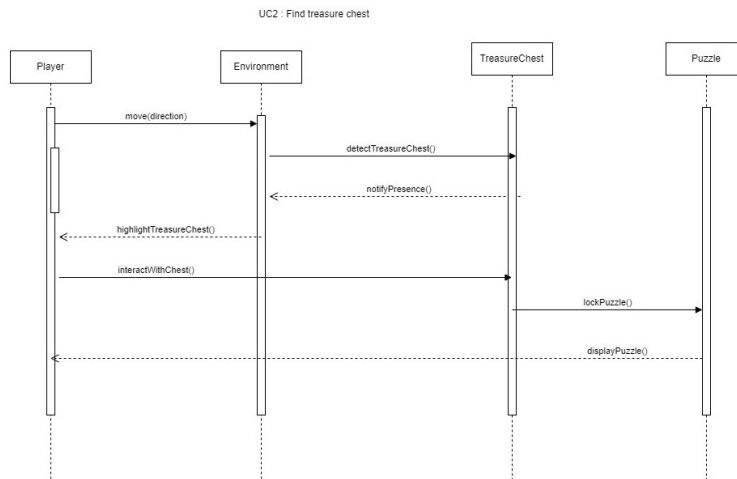
3.2.3 Class-level Sequence Diagram

3.2.3.1 Sequence diagram 1



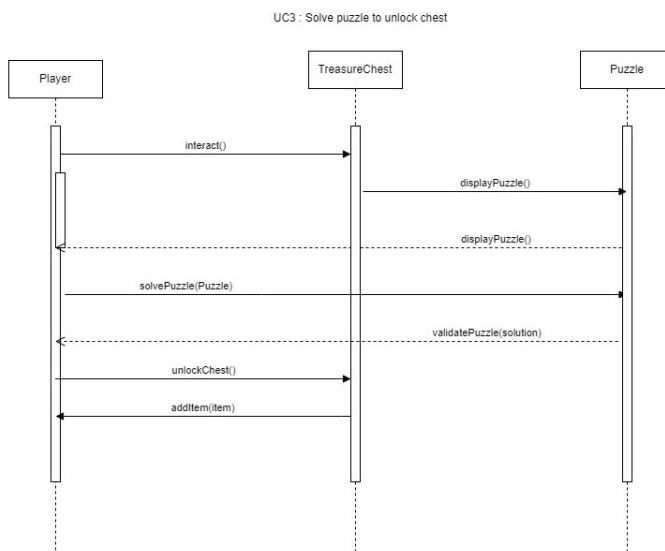
Sequence diagram 1

3.2.3.2 Sequence diagram 2



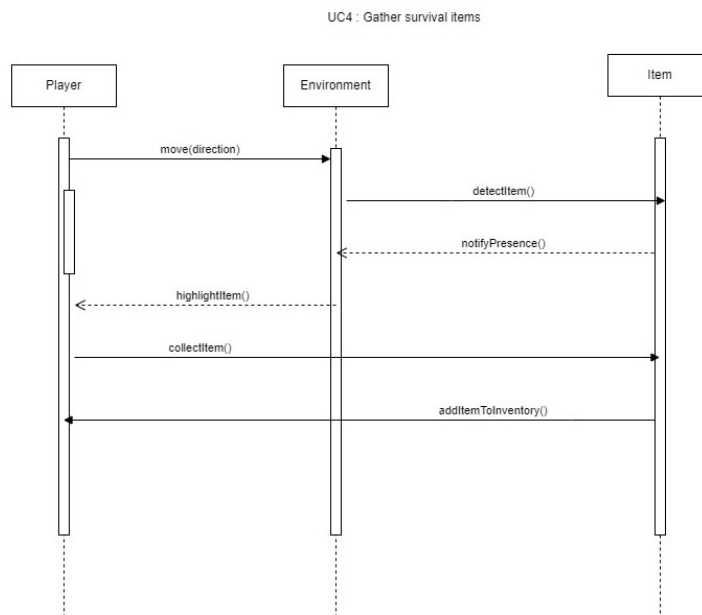
Sequence diagram 2

3.2.3.3 Sequence diagram 3



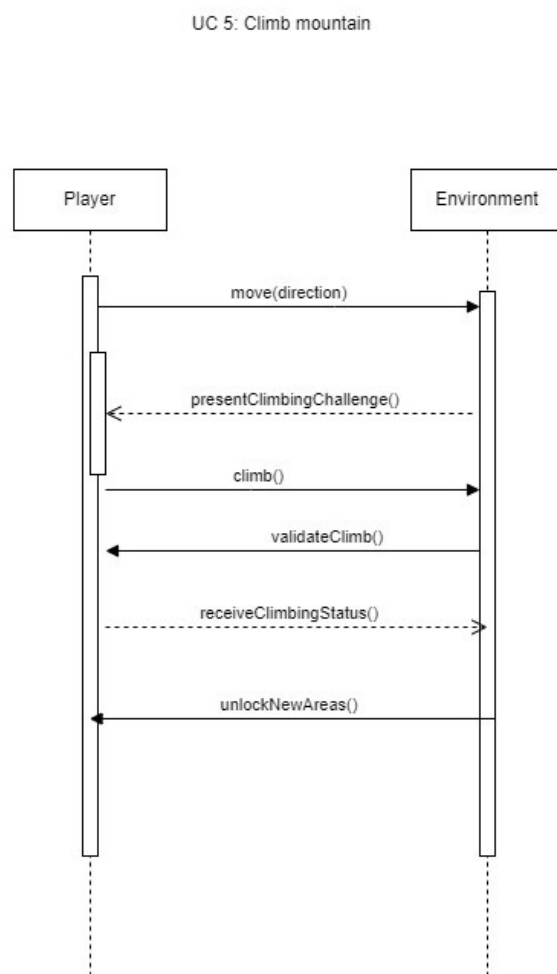
Sequence diagram 3

3.2.3.4 Sequence diagram 4



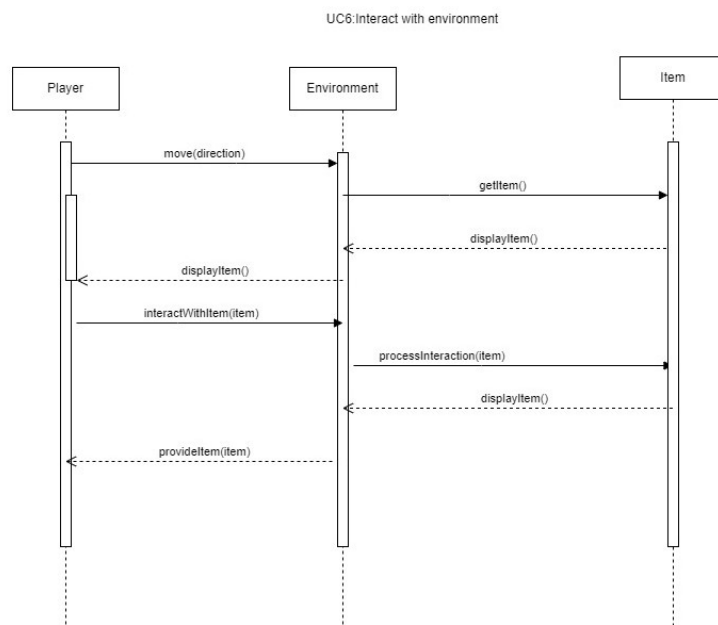
Sequence diagram 4

3.2.3.5 Sequence diagram 5



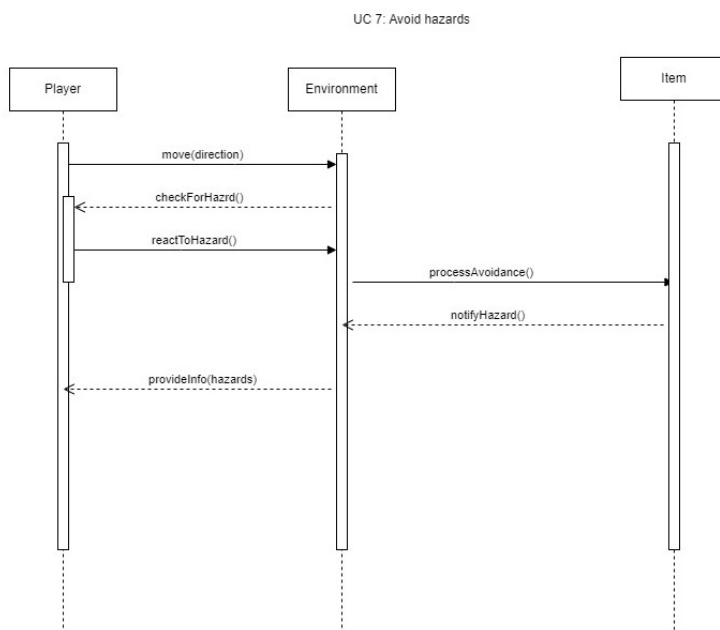
Sequence diagram 5

3.2.3.6 Sequence diagram 6



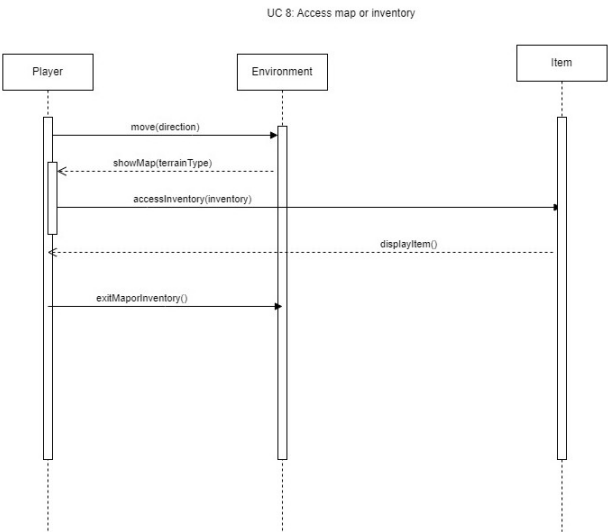
Sequence diagram 6

3.2.3.7 Sequence diagram 7



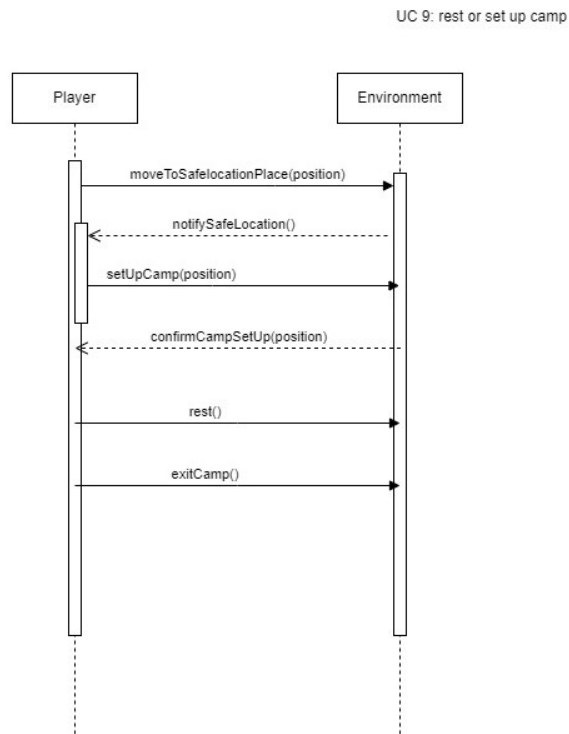
Sequence diagram 7

3.2.3.8 Sequence diagram 8



Sequence diagram 8

3.2.3.9 Sequence diagram 9



Sequence diagram 9

3.3 Data Design

The VR scavenger hunt system's information domain is organized into data entities that stand in for the key parts of the game, such as players, objects, treasure chests, puzzles, and ambient features. Specific properties and associations are defined for each object to enable effective data processing, organization, and storage.

3.3.1 Major Data Entities

3.3.1.1 Player

Attributes: Player ID, Name, Current Location, Inventory (list of Item IDs), Progress (current quest or level).

Storage: Player data is stored in a relational database table called Players, which maintains unique player records and their associated progress.

3.3.1.2 Item

Attributes: Item ID, Name, Type (e.g., food, tool), Description, Quantity, Location (current or initial).

Storage: Items are stored in a table called Items, which allows for efficient retrieval and manipulation of item-related data.

3.3.1.3 Puzzle

Attributes: Puzzle ID, Description, Difficulty Level, Solution (encrypted), Associated TreasureChest ID.

Storage: Puzzle data is organized in a table called Puzzles, linking each puzzle to its corresponding treasure chest.

3.3.1.4 TreasureChest

Attributes: Chest ID, Location, Status (locked/unlocked), Associated Puzzle ID.

Storage: Treasure chests are maintained in a table named TreasureChests, which tracks the status and associations with puzzles.

3.3.1.5 Hazard

Attributes: Hazard ID, Type (e.g., environmental danger, enemy), Location, Severity Level.

Storage: Hazards are recorded in a table called Hazards, detailing their type and effects on the player.

3.3.2 Data Storage and Processing

The system stores and retrieves data elements in an organized manner using a relational database management system (RDBMS). Foreign key relationships link each table, guaranteeing data integrity and allowing for sophisticated queries to extract insights like inventory control and player progress.

3.3.2.1 Database Management System

PostgreSQL will be used by the system as the database management system, enabling comprehensive data administration and handling. Each table in the database—Players, Items, Puzzles, Treasure Chests, and Hazards—is intended to store the characteristics of a different type of data entity. SQL queries are used to retrieve and manipulate data, enabling effective data operations during games and interactions.

Data Storage:

We will use Firebase for user authentication and real-time data synchronization in addition to PostgreSQL. Cloud-based storage solutions can be seamlessly integrated with Firebase, enabling the safe storing of inventory data and user progress.

3.3.3 Data Organization

The object-oriented design of the system is reflected in the organization of data into entities, where each class is mapped to a database table. This mapping makes it possible for the application logic and data storage to integrate seamlessly, making tasks like updating player progress, adding things to inventory, and solving puzzles quickly and easily.